

[Bài đọc] Đa ngôn ngữ

1. Khái niệm

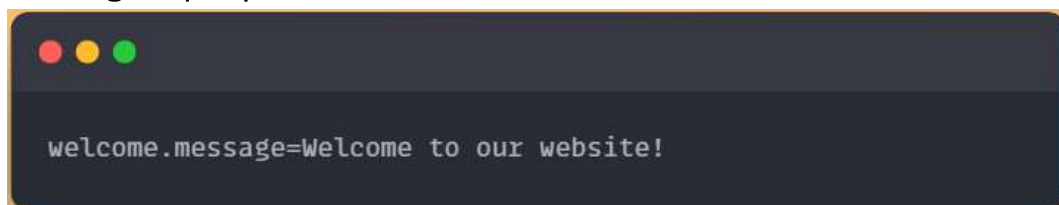
- **Đa ngôn ngữ** trong ứng dụng web (Internationalization - i18n) là khả năng hiển thị nội dung giao diện và thông báo theo nhiều ngôn ngữ khác nhau, tùy vào ngôn ngữ mà người dùng lựa chọn hoặc thiết bị/browser của họ
- Trong **Spring MVC**, việc hỗ trợ i18n thường được thực hiện bằng cách:
 - Lưu trữ các nội dung cần dịch trong file **.properties** theo từng ngôn ngữ
 - Cấu hình **message source** để Spring MVC có thể đọc các file này
 - Sử dụng **LocaleResolver** để xác định ngôn ngữ hiện tại
 - Cho phép thay đổi ngôn ngữ thông qua tham số URL, cookie hoặc session

2. Cấu trúc thư mục file ngôn ngữ

- Ví dụ cấu trúc file resource:

```
src/main/resources/
├─ messages.properties      (ngôn ngữ mặc định, ví dụ: English)
├─ messages_vi.properties   (Tiếng Việt)
└─ messages_fr.properties   (Tiếng Pháp)
```

- Ví dụ nội dung:
 - messages.properties



- messages_vi.properties

```
welcome.message=Chào mừng bạn đến với trang web của chúng tôi!
```

3. Cấu hình trong Spring MVC

- Cấu hình MessageSource

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Bean
    public ResourceBundleMessageSource messageSource() {
        ResourceBundleMessageSource messageSource = new ResourceBundleMessageSource();
        messageSource.setBasename("messages"); // tên file không kèm đuôi .properties
        messageSource.setDefaultEncoding("UTF-8");
        return messageSource;
    }
}
```

- Cấu hình LocaleResolver và
LocaleChangeInterceptor

```
@Bean
public LocaleResolver localeResolver() {
    SessionLocaleResolver slr = new SessionLocaleResolver();
    slr.setDefaultLocale(Locale.ENGLISH);
    return slr;
}

@Bean
public LocaleChangeInterceptor localeChangeInterceptor() {
    LocaleChangeInterceptor lci = new LocaleChangeInterceptor();
    lci.setParamName("lang"); // tham số URL để đổi ngôn ngữ
    return lci;
}

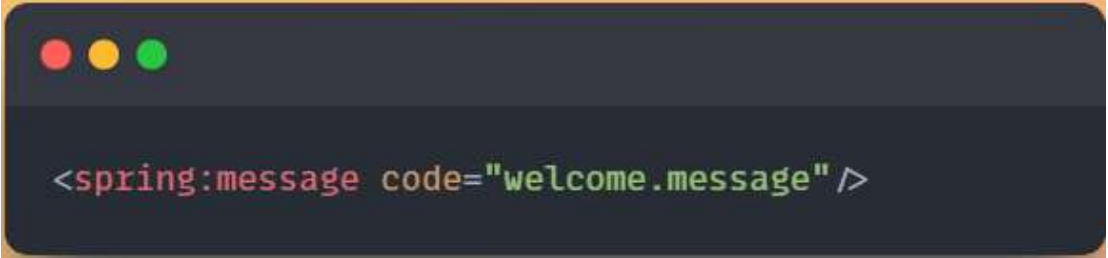
@Override
public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(localeChangeInterceptor());
}
```

4. Cách sử dụng trong View (Thymeleaf / JSP)

- Nếu dùng Thymeleaf

```
<p th:text="#{welcome.message}"></p>
```

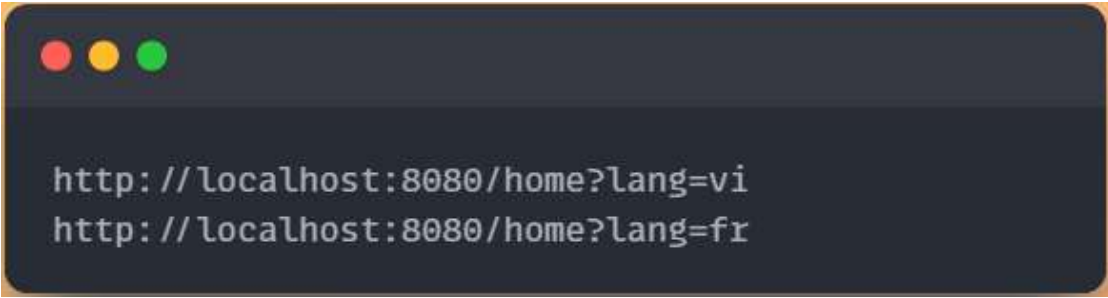
- Nếu dùng JSP



```
<spring:message code="welcome.message" />
```

5. Thay đổi ngôn ngữ

- Giả sử bạn cấu hình `localeChangeInterceptor` với param `lang`,
- bạn có thể thay đổi ngôn ngữ bằng URL:



```
http://localhost:8080/home?lang=vi  
http://localhost:8080/home?lang=fr
```

6. Tóm tắt

- Tạo file `messages_XX.properties` cho từng ngôn ngữ
- Cấu hình `messageSource` để Spring đọc file
- Cấu hình `LocaleResolver` để lưu ngôn ngữ hiện tại (session, cookie, header)
- Thêm `LocaleChangeInterceptor` để bắt tham số đổi ngôn ngữ
- Gọi `message` trong view bằng cú pháp `#{...}` (Thymeleaf) hoặc `<spring:message ...>` (JSP)